

Registre de classe sur blockchain — Rapport technique

Projet : stockage et partage des supports et notes d'une classe sur une blockchain privée **Plateforme retenue** : Hyperledger Besu (open source) hébergé sur Microsoft Azure **Langage principal** : Python (application) ; Solidity (smart contract) **Date de l'étude** : 5 septembre 2026

1. Résumé

Ce projet met en pratique la technologie blockchain pour gérer les supports pédagogiques (cours, TP, examens, corrections) et les notes d'une classe, en respectant quatre règles d'accès :

1. la description et l'organisation de la classe sont publiques ;
2. les cours et TP ne sont visibles que par les étudiants inscrits ;
3. les examens et corrections ne sont disponibles que 24 h après la date d'examen, et seulement pour les inscrits ;
4. chaque étudiant n'a accès qu'à ses propres notes.

Ces règles sont codées **dans un smart contract**, exécuté par un réseau privé de quatre nœuds Hyperledger Besu utilisant le consensus **QBFT**. Une application web Python (FastAPI) sert de passerelle : elle interroge le contrat au nom de l'utilisateur, conserve les fichiers volumineux sur disque et ancre leur empreinte SHA-256 sur la chaîne. L'ensemble se déploie avec Docker Compose sur une seule machine virtuelle Azure.

Le livrable est fonctionnel : le réseau produit des blocs, tolère la panne d'un validateur, et les quatre règles sont vérifiées par des tests automatisés et par un scénario de bout en bout.

2. Contexte et exigences

2.1 Rappel du principe

Une blockchain est un registre de transactions **immuable**, répliqué sur un réseau **P2P** de nœuds, chaque nœud appliquant les transactions validées par un **protocole de consensus** et regroupées en blocs chaînés par empreinte cryptographique. Les propriétés attendues par l'énoncé sont : transparence (chacun a une copie), immutabilité (écriture unique), distribution (pas de tiers de confiance) et validation par consensus global.

2.2 Exigences fonctionnelles

Réf.	Exigence	Traduction technique
E1	Description/organisation publiques	Lecture libre, sans authentification
E2	Cours/TP réservés aux inscrits	Contrôle <code>isEnrolled(msg.sender)</code>
E3	Examens/corrections 24 h après l'examen	<code>now ≥ examAt + 24 h</code> et inscrit
E4	Notes privées par étudiant	Lecture limitée à <code>msg.sender == étudiant</code>
E5	Immutabilité	Aucune modification/suppression ; versions chaînées

2.3 Exigences non fonctionnelles

- **Simplicité** : périmètre volontairement réduit à une classe.
- **Fournisseur gratuit ou en essai** : coût maîtrisé.
- **Reproductibilité** : démarrage local et déploiement scriptés.
- **Confidentialité** : les notes ne doivent pas fuiter entre étudiants.

3. État de l'art des plateformes (au 5 septembre 2026)

L'énoncé demande d'étudier les fournisseurs (AWS, Azure, IBM, etc.) et de choisir celui offrant un accès gratuit ou un essai, puis de concevoir la solution « des structures de données au consensus ». Nous comparons les offres managées et les plateformes open source auto-hébergeables.

3.1 Offres managées

Microsoft Azure. Microsoft a retiré **Azure Blockchain Service** le **10 septembre 2021** ; il n'existe plus d'offre « blockchain à smart contracts » managée chez Azure en 2026. Subsiste **Azure Confidential Ledger** : un registre append-only, inviolable, basé sur le Confidential Consortium Framework et l'informatique confidentielle. Il fournit immutabilité et preuve d'intégrité (reçus, arbre de Merkle) et des SDK .NET, Java, Python, JavaScript. **Limite majeure pour notre cas : il n'exécute pas de smart contracts** ; on ne peut donc pas y coder les règles d'accès (inscription, délai de 24 h, isolation des notes). Tarif indicatif : **0,124 USD/h par instance** plus **0,16 USD/Go/mois**, soit environ 90 USD/mois en fonctionnement continu.

Amazon Web Services. Amazon Managed Blockchain propose Hyperledger Fabric ainsi que l'accès à des réseaux Ethereum/Polygon/Bitcoin. La facturation est à la consommation (nœud à la seconde avec minimum, stockage, requêtes) : adaptée à la production, sans réel palier gratuit, et l'édition Fabric managée reste plus lourde à prendre en main qu'un réseau local.

IBM. La documentation officielle indique que **IBM Blockchain Platform 2.5.4 n'est plus publié** et renvoie vers le support Hyperledger Fabric. L'ancien produit n'est donc pas une voie actuelle avec essai simple pour un nouveau projet.

Oracle. Oracle Blockchain Platform (Fabric managé) est disponible mais payant (édition Standard à **0,215 USD/OCPU·h**, plus stockage), sans intérêt ici face à une solution open source gratuite.

3.2 Plateformes open source auto-hébergeables

Hyperledger Fabric. Blockchain permissionnée d'entreprise, très riche (canaux, politiques d'endorsement, données privées). Le chaincode s'écrit en **Go, Node.js ou Java** — pas en Python. La mise en place (peers, orderer, MSP, autorités de certification, canaux) est nettement plus complexe que nécessaire pour une seule classe.

Hyperledger Besu. Client Ethereum de niveau entreprise, open source (Apache 2.0). Il exécute des smart contracts **EVM/Solidity**, supporte des consensus **PoA QBFT/IBFT** adaptés aux réseaux privés (sans minage ni cryptomonnaie), se déploie en **Docker**, et s'interface parfaitement avec **web3.py** côté Python. QBFT est byzantin-tolérant : avec n validateurs il tolère $f = \lfloor (n-1)/3 \rfloor$ pannes, d'où **4 validateurs** pour tolérer 1 panne.

3.3 Comparatif de synthèse

Critère	Azure Conf. Ledger	AWS Managed BC	IBM BP	Oracle BP	Fabric (auto)	Besu (auto)
Smart contracts	Non	Oui	Oui	Oui	Oui (Go/JS/Java)	Oui (Solidity)
Langage app Python	Oui	Oui	Oui	Oui	Partiel	Oui (web3.py)
Consensus configurable/visible	Non	Partiel	Partiel	Partiel	Oui	Oui (QBFT)
Coût	~90 USD/mois	À l'usage	Plus publié	Payant	Gratuit	Gratuit
Complexité mise en œuvre	Faible	Moyenne	—	Moyenne	Élevée	Faible/Moyenne

Critère	Azure Conf. Ledger	AWS Managed BC	IBM BP	Oracle BP	Fabric (auto)	Besu (auto)
Hébergeable sur Azure	Natif	Non	Non	Non	Oui (VM)	Oui (VM)

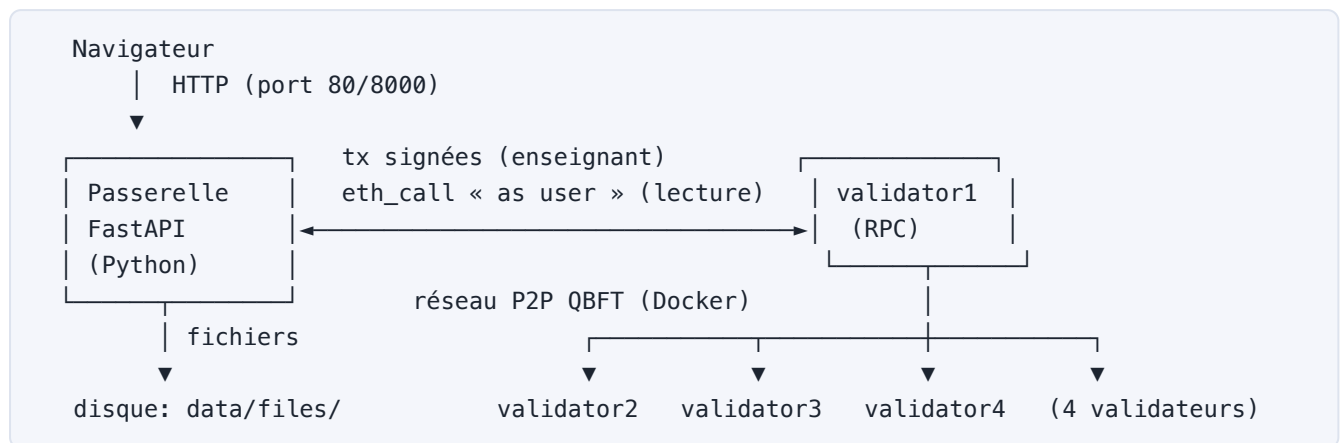
3.4 Décision

Nous retenons **Hyperledger Besu auto-hébergé sur une VM Azure**. Justification :

- c'est **gratuit** et open source (contrainte « accès gratuit ») ;
- il permet d'écrire les règles dans un **smart contract** (les offres sans contrats, comme Azure Confidential Ledger, ne peuvent pas exprimer E2–E4) ;
- il rend le **consensus** explicite et démontrable (QBFT, tolérance de panne), ce qui répond à la demande « de la structure de données au consensus » ;
- **Python** reste le langage de l'application ;
- le **déploiement Docker** est simple et reproductible.

La préférence pour Azure est satisfaite au niveau de l'**hébergement** : faute de service blockchain managé Azure adéquat en 2026, Azure fournit la VM et l'infrastructure, ce que l'énoncé autorise explicitement (« any open source blockchain platform »).

4. Architecture de la solution



Principe clé. Les décisions d'accès sont prises **par le contrat**. En lecture, la passerelle appelle les fonctions `view` en positionnant l'adresse de l'utilisateur comme `msg.sender` (paramètre `from` d'`eth_call`, qui ne requiert pas de signature). Le contrat applique alors E1–E4 ; la passerelle ne fait que refléter le résultat et présente des messages clairs. Le port RPC n'est jamais exposé publiquement.

4.1 Données on-chain / off-chain

- **Sur la chaîne** : description/organisation, liste des inscrits, métadonnées des supports (type, titre, date d'examen, version précédente), **empreinte SHA-256** de chaque fichier, et notes par étudiant.
- **Hors chaîne** : les fichiers eux-mêmes, sur le disque de la VM (`data/files/`). La chaîne garantit leur intégrité via le hash ; au téléchargement, la passerelle recalcule le SHA-256 et le compare à celui ancré.

Ce partage évite de stocker des binaires volumineux sur la blockchain tout en conservant la preuve d'intégrité et le contrôle d'accès on-chain.

5. Structures de données et smart contract

Le contrat `ClassLedger.sol` (Solidity 0.8.24) contient tout le modèle et toutes les règles.

5.1 Modèle

```
enum Kind { LECTURE, LAB, EXAM, CORRECTION }

struct Material {
    Kind    kind;           // type de support
    string  title;          // titre
    string  fileRef;        // référence opaque du fichier sur disque
    bytes32 fileHash;       // SHA-256 du fichier (intégrité)
    uint64  examAt;         // date d'examen (EXAM/CORRECTION)
    bool    hasPrev;        // s'agit-il d'une révision ?
    uint256 prevId;         // version précédente (conservée)
    uint64  createdAt;
}
```

L'état comprend l'adresse de l'enseignant, la description/organisation publiques, le tableau `materials`, la table `isEnrolled`, et les notes (`grade`, `gradePublished`) lisibles seulement par leur propriétaire ou l'enseignant.

5.2 Règles (extraits)

Contrôle central de visibilité :

```
function _visible(Material storage m) internal view returns (bool) {
    if (msg.sender == teacher) return true;           // l'enseignant voit tout
    if (!isEnrolled[msg.sender]) return false;        // E2 : inscrits seulement
    if (m.kind == Kind.EXAM || m.kind == Kind.CORRECTION)
        return block.timestamp >= uint256(m.examAt) + EXAM_DELAY; // E3 : +24 h
    return true;                                       // cours & TP
}
```

Notes privées (E4) :

```
function myGrade() external view returns (bool, string memory) {
    return (gradePublished[msg.sender],
            gradePublished[msg.sender] ? grade[msg.sender] : "");
}

function gradeOf(address s) external view returns (bool, string memory) {
    require(msg.sender == teacher || msg.sender == s, "forbidden");
    ...
}
```

Écritures réservées à l'enseignant via le modificateur `onlyTeacher`. Les événements (`MaterialAdded`, `GradePublished`) sont émis **sans** valeur de note.

5.3 Immutabilité (E5)

Aucune fonction ne modifie ni ne supprime un support. Une correction ou une nouvelle version est **ajoutée** avec `hasPrev = true` et `prevId` pointant vers l'entrée précédente, qui reste intacte et consultable. L'historique est

donc préservé, conformément au principe « write once ».

6. Consensus QBFT

6.1 Fonctionnement

QBFT (Quorum Byzantine Fault Tolerance) est un consensus **Proof of Authority** : un ensemble de validateurs autorisés proposent et valident les blocs à tour de rôle. Un bloc est ajouté lorsqu'une **super-majorité ($\geq 2/3$)** des validateurs le signe. Il n'y a ni minage ni cryptomonnaie : le coût opérationnel est celui d'un système distribué classique, et la finalité est immédiate (pas de fork probabiliste).

6.2 Dimensionnement

Avec `n` validateurs, QBFT tolère

$$f = \lfloor (n - 1) / 3 \rfloor$$

pannes byzantines. Pour tolérer **1** défaillance, il faut `n = 4` ($\lfloor 3/3 \rfloor = 1$), d'où notre réseau de **quatre validateurs**. Si plus d'un tiers des validateurs cesse de participer, la production de blocs s'arrête jusqu'au rétablissement — c'est le comportement sûr attendu.

6.3 Paramètres retenus

Chaîne privée `chainId = 1337`, **gas à zéro** (`--min-gas-price=0`, transactions à `gasPrice = 0`), temps de bloc `blockperiodseconds = 2`. L'absence de gas payant simplifie la démonstration (aucun approvisionnement de compte nécessaire) tout en conservant un EVM complet.

7. Sécurité et confidentialité

7.1 Contrôles

- **Rôles** : seul l'enseignant détient une clé privée EVM et peut écrire. Les étudiants sont en lecture seule, via la passerelle.
- **Authentification web** : comptes locaux (SQLite), mots de passe hachés en **Argon2**, session par cookie signé (HttpOnly, SameSite).
- **Intégrité des fichiers** : SHA-256 ancré on-chain, revérifié au téléchargement.
- **Surface réseau** : seul le port web est exposé ; le RPC Besu et le P2P restent internes au réseau Docker.

7.2 Limite de confidentialité (assumée)

Sur une blockchain, **tout opérateur de nœud peut lire l'état brut** du registre. La confidentialité des notes ne repose donc pas sur la chaîne elle-même mais sur deux éléments : la **passerelle** (qui n'expose que la note de l'utilisateur authentifié, en appelant le contrat avec son adresse) et le **RPC non public**. Dans notre démonstration, les quatre validateurs appartiennent au même administrateur ; ils sont dans le périmètre de confiance. Une exigence de confidentialité forte entre organisations relèverait de mécanismes dédiés (données privées Fabric, informatique confidentielle, chiffrement applicatif), hors périmètre de ce projet. Cette limite est explicitement documentée, comme il se doit.

8. Implémentation

8.1 Pile technique

Couche	Technologie
Blockchain	Hyperledger Besu 26.2.0, QBFT, Docker Compose
Contrat	Solidity 0.8.24 (EVM <i>berlin</i>)
Application	Python 3.11, FastAPI, Jinja2, web3.py 7
Authentification	SQLite, Argon2 (pwndlib)
Tests	pytest, EVM en mémoire (eth-tester/py-evm)

8.2 Passerelle

L'application expose : page publique, connexion, tableaux enseignant/étudiant, téléchargement contrôlé et vue « preuve » (bloc courant, pairs, validateurs). Les écritures sont des transactions signées par l'enseignant ; les lectures sont des `eth_call` portant l'adresse de l'utilisateur. Lors d'un dépôt, le fichier est d'abord écrit sur disque, son hash calculé, puis la transaction envoyée ; en cas d'échec de la transaction, le fichier est supprimé pour garder disque et chaîne cohérents.

8.3 Difficulté technique notable : version d'EVM

Solidity $\geq 0.8.20$ cible par défaut l'EVM *Shanghai* et émet l'opcode `PUSH0`. Or notre genesis Besu n'active que le palier *Berlin* : `PUSH0` y est invalide, ce qui faisait échouer le déploiement du contrat (transaction en échec, tout le gas consommé, aucun code déposé). La correction a consisté à **compiler le contrat pour l'EVM *berlin***, afin d'aligner le bytecode sur la chaîne — un choix cohérent avec une chaîne privée simple, sans EIP-1559. Cet incident illustre l'importance de faire correspondre la cible de compilation au palier de fork du réseau.

9. Tests et validation

9.1 Tests unitaires du contrat

Sept tests (pytest sur EVM en mémoire) couvrent les règles et l'intégrité :

- écriture réservée à l'enseignant ;
- cours/TP invisibles pour un non-inscrit, visibles après inscription ;
- examen/correction refusés avant 24 h, autorisés après (avance de l'horloge) ;
- note publiée visible par son propriétaire, **inaccessible** à un autre étudiant ;
- révision ajoutée sans écraser l'original.
- nom de fichier assaini et empreinte SHA-256 préservée.

Résultat : **7 tests réussis**, en local et dans le conteneur (`make test`).

9.2 Scénario de bout en bout

`make smoke` exécute, sur le réseau réel : vérification du **nombre de pairs (3)** et du déploiement, connexion enseignant, dépôt d'un cours et d'un examen verrouillé, publication d'une note, puis connexion étudiant pour vérifier la visibilité du cours, le **verrou 24 h** de l'examen, la lecture de **sa seule** note, et un **téléchargement vérifié par hash**. Le téléchargement de l'examen verrouillé est refusé. Tous les contrôles passent.

9.3 Tolérance de panne

Avec un validateur arrêté, une vraie transaction de mise à jour a été finalisée avec `status = 1` au bloc 39 par les trois validateurs restants : le réseau tolère bien **une** défaillance, conformément à QBFT.

10. Déploiement Azure et coûts

Le déploiement cible **une** VM Ubuntu (Standard_B2ms, 2 vCPU / 8 Go) provisionnée par script Azure CLI ; Docker est installé via cloud-init. Le code est copié par `rsync`, puis `make up && make deploy && make seed` démarrent la pile. Seul le port **80** est ouvert ; SSH est restreint à l'IP de l'administrateur ; le RPC reste interne.

Maîtrise du coût. L'API officielle Azure Retail Prices indique **0,0944 USD/h** pour une B2ms Linux en France Central au 5 septembre 2026, soit environ 68,91 USD pour 730 h continues, hors disque et réseau. En **désallouant** la VM hors démonstration (`stop.sh`), la facturation de calcul s'arrête ; le coût du projet se limite alors à quelques euros. Le script crée un budget mensuel ; son seuil de notification email sera finalisé dans le portail lors de la recette Azure. `destroy.sh` supprime l'ensemble.

11. Résultats

- Réseau QBFT à 4 validateurs fonctionnel, blocs finalisés toutes les ~2 s.
- Contrat déployé, quatre règles d'accès **vérifiées** par tests et scénario.
- Intégrité des fichiers garantie par empreinte on-chain.
- Tolérance à la panne d'un validateur démontrée.
- Déploiement local reproductible ; scripts Azure prêts pour la recette sur l'abonnement cible.

12. Limites et perspectives

- **Confidentialité** limitée par nature (cf. §7.2) ; une version inter-organisations nécessiterait des données privées ou du chiffrement applicatif.
- **Topologie de démonstration** : les 4 validateurs partagent une VM ; en production ils seraient répartis sur plusieurs hôtes/organisations.
- **Périmètre** : une classe, deux étudiants fictifs. L'extension multi-classes, la rotation de validateurs par vote QBFT ou un stockage de fichiers répliqué sont des pistes naturelles.

13. Conclusion

Le projet démontre qu'une blockchain permissionnée légère (Hyperledger Besu, QBFT) répond précisément au besoin : les structures de données et les quatre règles d'accès sont portées par un smart contract, le consensus est explicite et tolérant aux pannes, et l'application Python reste simple. Le choix d'héberger une plateforme open source sur Azure concilie la préférence pour Azure et la contrainte de gratuité, tout en gardant une mise en œuvre minimale et reproductible, fidèle à l'énoncé.

Références

Sources officielles consultées le **5 septembre 2026** :

- [Hyperledger Besu — QBFT](#).
- [Microsoft Learn — Azure Confidential Ledger](#).
- [Azure — Tarification Confidential Ledger](#).
- [Azure — Retrait d'Azure Blockchain Service](#).
- [Azure Retail Prices API](#) — requête B2ms Linux, France Central.
- [AWS — Amazon Managed Blockchain Pricing](#).
- [IBM — Blockchain Platform 2.5.4](#) (« no longer published »).
- [Oracle Cloud — Price List, Blockchain Platform](#).
- [Hyperledger Fabric — Introduction](#) et [chaincode](#).
- [web3.py](#) et [FastAPI](#).